VexFlow Basics: (see resources for tutorial)

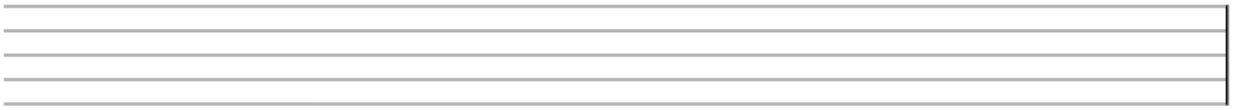
Hello World:

The starting point for VexFlow as used in this project is the Factory object. The Factory can be understood as the “home base” for VexFlow, most features are built off of the Factory. The Factory also specifies a renderer object and the element id to attach it to in the HTML.

The most important object built off of the Factory is the EasyScore object. VexFlow is very hands on, and objects such as notes have to be created by hand. This can get tedious, so EasyScore was created to take higher level instructions written in a simpler format and translate them into lower level VexFlow code.

The last fundamental object is the System object. A system is a group of Stave objects grouped together for formatting, and attaching objects to a System is what allows them to finally be rendered. With this basic understanding, here is our version of “hello world”, an empty bar of music:

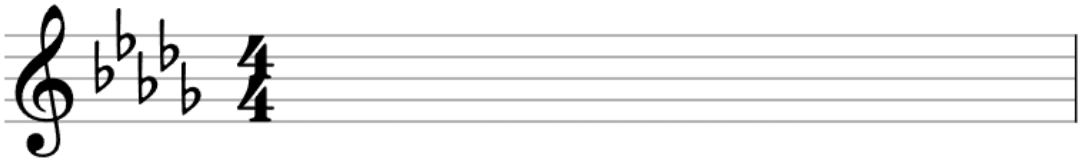
```
import { Factory } from "https://cdn.jsdelivr.net/npm/vexflow@4.2.2/build/esm/entry/vexflow.js";

const vf = new Factory({ renderer: { elementId: "output", width: 4000, height: 150 } });
const score = vf.EasyScore();

const system = vf.System();
system.addStave({
    voices: []
});


vf.draw();
```



Stave Modifiers:

This piece of music is obviously not very interesting, so let’s add some typical features of a measure: time signature, key signature, and clef. Stave objects have a variety of modifiers which can be added using the addModifier() method, but these three features are so common that they have their own methods: addTimeSignataure(), addKeySignature(), and addClef()

```
const system = vf.System();
system.addStave({
    voices: []
}).addTimeSignature('4/4').addKeySignature('Db').addClef('treble');
```

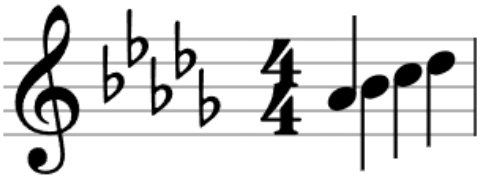


We can also add measure numbers by including .setMeasure() in our modifiers.

Voices:

Now we're getting somewhere. But we're still missing the most important part: the notes themselves. To add these, we have to add a "voice" to our staff. A voice simply contains a group of "Tickables", which are items that have a duration and can be placed on the stave, for example, notes and lyrics. Here's where EasyScore is super helpful; we can use the EasyScore.notes() method to create these Tickables from a string of notes, which can then be turned into a voice:

```
const system = vf.System();
system.addStave({
    voices: [score.voice(score.notes('a4/q, b4/q, c5/q, d5/q'))]
}).addTimeSignature('4/4').addKeySignature('Db').addClef('treble');
```

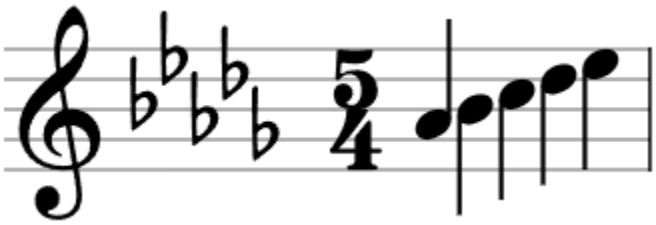


Before moving on, it's worth taking a moment to understand the EasyScore.notes() method. As you can see, each note is specified by the note name and octave, followed by the duration. Valid durations are as follows (long to short): 'w', 'h', 'q', '8', '16', '32', '64', and '128'. Durations also cascade, so if specified, that duration will be applied to all subsequent notes until a new duration is specified. To draw rests, simply add /r at the end of the note. (ex: 'a4/q/r') To create dotted rhythms, after the duration, add a number of periods equal to the number of dots. (ex: 'a4/q.') To add accidentals, after specifying the note name, include '#' for sharp, 'b', for flat, and 'n' for natural. (This is expandable for multiple sharps or flats)

The EasyScore.voice() function also deserves some attention. Throughout VexFlow and EasyScore, most methods have a variety of options that alter the results. The .voice() method

has a particularly important option: time. By default, every measure is assumed to be in 4/4 time, and adding a new key signature does not change this fact, only the visual element. To create a measure in a different time signature, it must be specified, or the resulting voice will either have too few or too many notes, and it won't render properly. For example:

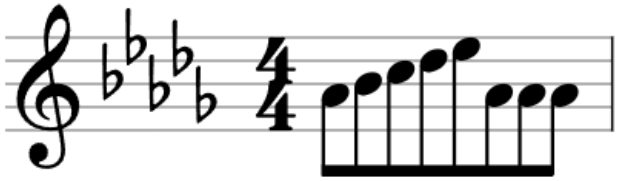
```
const system = vf.System();
system.addStave({
    voices: [score.voice(score.notes('a4/q, b4, c5, d5, e5'), { time: '5/4' })]
}).setTimeSignature('5/4').addKeySignature('Db').addClef('treble');
```



Note Groupings:

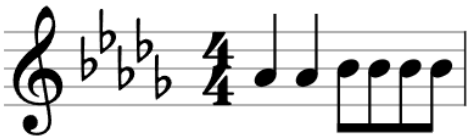
Now let's start making more complex music, beginning with beamed notes. Making beamed notes is relatively simple; all you need to do is wrap a call to EasyScore.notes() with a call to EasyScore.beam(). The only catch is that each note is formatted individually by EasyScore.notes(), and the stem directions may not line up for an aesthetically pleasing beam. To fix this, we can use the "autoStem" option of beam():

```
const system = vf.System();
system.addStave({
    voices: [score.voice(score.beam(score.notes('a4/8, b4, c5, d5, e5, a4, a4, a4'), { autoStem: true }))]
}).setTimeSignature('4/4').addKeySignature('Db').addClef('treble');
```



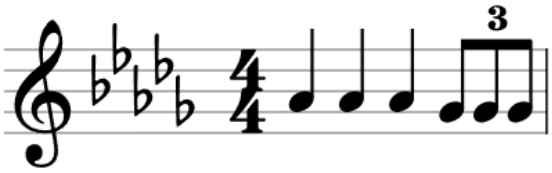
But what if we only want some of our notes to be beamed? That's where the concat() method comes in. With it, we can mash together multiple calls to EasyScore.notes():

```
const system = vf.System();
system.addStave({
    voices: [score.voice(
        score.notes('a4/q, a4')
        .concat(score.beam(score.notes('b4/8, b4, b4, b4')))
    )]
}).setTimeSignature('4/4').addKeySignature('Db').addClef('treble');
```



With beams and concatenation under our belt, tuplets are very easy. Just as we wrapped notes with a beam, we wrap a beam with a tuplet. Unfortunately, tuplets automatically render with a ratio of the number of notes in the tuplet to the number of notes that would normally fit in that space. To get around this, we use the “ratioed” option:

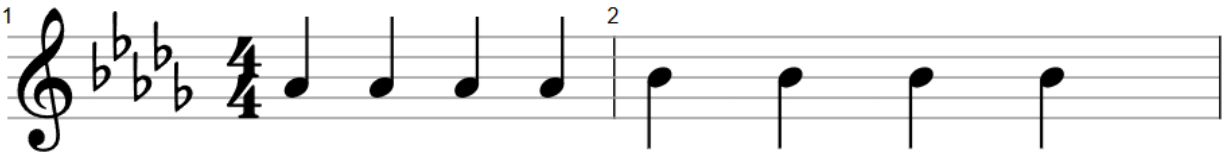
```
const system = vf.System();
system.addStave({
    voices: [score.voice(
        score.notes('a4/q, a4, a4')
        .concat(score.tuplet(score.beam(score.notes('g4/8, g4, g4')), { ratioed: false }))
    )]
}).setTimeSignature('4/4').addKeySignature('Db').addClef('treble');
```



System Modifiers:

Up until now, we’ve been working with single measures, but how do we add new ones? We just have to create another system! However, if we just make two systems as we’ve been doing, they’ll render on top of each other. We need a way to specify the location. Luckily, the System constructor lets us specify both the width and position of a measure:

```
const system1 = vf.System({ width: 300, x: 10});
system1.addStave({
    voices: [score.voice(score.notes('a4/q, a4, a4, a4'))]
}).setMeasure(1).setTimeSignature('4/4').addKeySignature('Db').addClef('treble');

const system2 = vf.System({width: 300, x: 310});
system2.addStave({
    voices: [score.voice(score.notes('b4/q, b4, b4, b4'))]
}).setMeasure(2);
```



Other Objects Used:

Barline/BarlineType:

Sometimes, we need to add or modify the barlines in a measure, such as for key changes, repeats, or the end of a piece of music. There are two ways to add a barline, and we will use both. First, update your imports:

```
import { Factory, Barline, BarlineType } from
```

The first way is to manually add a new Barline:

```
const system = vf.System();
system.addStave({
    voices: []
}).addModifier(new Barline(BarlineType.DOUBLE));
```

Note the use here of the enum BarlineType, which is one of the few enums that has decent documentation to explain the different values. By default, this goes on the leftmost end of the stave, but each stave modifier has a position value that can be adjusted to change this. We can use this default to place barlines like BarlineType.DOUBLE at the beginning of a bar, even when they're only supposed to render at the end of a bar. The other way to add a Barline is to set the barline that should appear at the beginning or end of a bar:

```
const system = vf.System();
system.addStave({
    voices: [],
})
.setBegBarType(BarlineType.REPEAT_BEGIN)
.setEndBarType(BarlineType.DOUBLE);
```

Here's the problem. These two functions are very specific as to which barline types they can display. For instance, you can't display and ending repeat at the beginning of a measure. However, these allowed types aren't specified. If a barline doesn't appear, this is likely the

problem. You can solve it by going into the stave's modifiers and manually changing the type, or by using the first approach.

Curve:

Next up, slurs and ties! We can make these fairly easily using the Curve object. As always with these added features, we have to start by importing Curve:

```
import { Factory, Curve } from
```

So, how do we create a Curve? It's fairly simple. We have to specify the starting and ending points, and then we have at our disposal a few customizable options:

```
const system = vf.System({width: 300});
var notes = score.notes('a4/q, a4, a4, a4');
system.addStave({
    voices: [score.voice(notes)],
});

var slur = new Curve(
    notes[1], notes[3], {
        cps: [
            {x: 0, y: 10},
            {x: 0, y: 10}
        ],
        y_shift: 15
    }
);

vf.draw();

slur.setContext(vf.getContext());
slur.draw();
```

Now, a lot is going on here, so let's break it down. The first new thing is fairly straightforward, which is just creating the notes separately from the voice. This is so we can have easy access to the notes when indexing our slur, and it often improves readability. The other way to access these notes would be to access the system's partVoices and find the 'tickables' array. Then we create the slur, specifying the start and end notes of the slur, the cps (which are two points on the bezier curve we can mess with to modify the shape of the curve), and the y_shift, which just moves it up and down. Once we have everything set up, we can call vf.draw(), except the slur has not been attached to the stave, so it won't be rendered. Unfortunately, the stave must be rendered first so the slur knows where to position itself, so we have to render all slurs after the measure they belong to. We do this by setting the render context of the slur based on our factory's render context and calling draw on the slur itself.

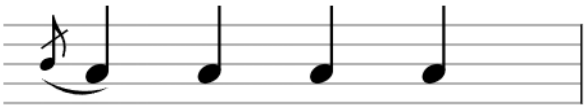
GraceNote/GraceNoteGroup:

Time for grace notes. As always, we start with the import:

```
import { Factory, GraceNote, GraceNoteGroup } from
```

The remaining process isn't too bad, we just need to specify which note value the grace note is, how long it should be, and create a new modifier for the note we want to attach it to:

```
const system = vf.System({width: 300});
var notes = score.notes('a4/q, a4, a4, a4');
system.addStave({
    voices: [score.voice(notes)],
});

var grace = new GraceNote({
    keys: ['b/4'],
    duration: '8',
    slash: true
});

var group = new GraceNoteGroup([grace], true);
notes[0].addModifier(group);
```



Note that the format for the key is slightly different than what we're used to with EasyScore. The group, as you can see, takes in an array of GraceNote objects, in the case multiple are needed for a single note, as well as a boolean value. This value specifies whether or not the slur between the two notes should be rendered.

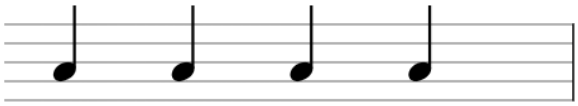
TextNote:

And what would a piece for vocalists be without lyrics? Once again, let's get our new object imported:

```
import { Factory, TextNote } from
```

The basic idea behind TextNotes is that they operate just like regular notes, and we need to create a voice containing our lyrics. Importantly, this means we have to pay attention to their durations

```
const system = vf.System({width: 300});
var notes = score.notes('a4/q, a4, a4, a4');

var lyric1 = new TextNote({
    text: "do",
    duration: 'h',
    line: 11
}).setJustification(TextNote.Justification.CENTER);
var lyric2 = new TextNote({
    text: "re",
    duration: 'h',
    line: 11
}).setJustification(TextNote.Justification.CENTER);

var lyrics = score.voice([lyric1, lyric2]);

console.log(lyrics);

system.addStave({
    voices: [score.voice(notes), lyrics],
});
```



do re

Each TextNote has the text it will display, its duration (for formatting purposes), and a line that shows where the text will be rendered. A higher line number will put the text further down the stave. We also set the justification to center, because by default, lyrics are slightly off center. All that's left to do then is create a voice containing our lyrics and add it to the voices option of our stave.

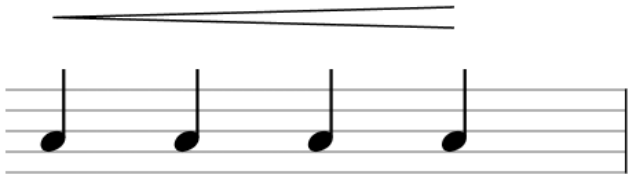
StaveHairpin:

Now it's time for dynamic markings. We'll do this in two sections. First up are hairpins, things like crescendo and decrescendo. Let's begin with the import:

```
import { Factory, StaveHairpin } from
```

Hairpins operate similarly to slurs/ties, in that we specify a starting and ending note, but with the added bonus of specifying a type. Similarly, we also have to attach the context and draw it separately. If the type we set is a 1, the hairpin will render as a crescendo; if it's 2, the hairpin will render as a decrescendo. Lastly, we have the ability to modify the y_shift (similar to most elements) in the render_options. In the following example I've used a negative offset to place the hairpin above the stave:

```
var notes = score.notes('a4/q, a4, a4, a4');
const system = vf.System({width: 300});
system.addStave({
    voices: [score.voice(notes)]
});

var crescendo = new StaveHairpin({
    first_note: notes[0],
    last_note: notes[3]
}, 1);
crescendo.render_options.y_shift = -110;
crescendo.setContext(vf.getContext());

vf.draw();
crescendo.draw();
```



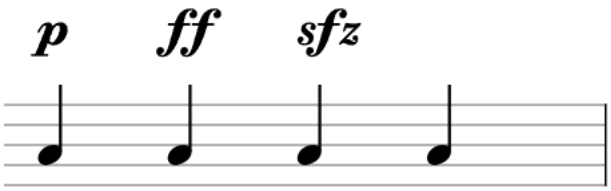
TextDynamics:

With hairpins working, the next step is to get dynamic markings, such as pianissimo, piano, forte, fortissimo, and sforzando, up and running. Starting once again with the import:

```
import { Factory, TextDynamics } from
```

Conveniently, these work almost the same as lyrics; we specify the text and the duration, but rather than rendering as text, using a TextDynamics object will ensure the text is rendered as proper dynamic notation. By default, these render above the stave:

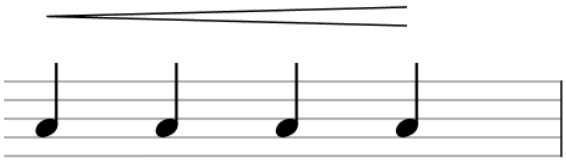
```
var piano = new TextDynamics({
    text: 'p',
    duration: 'q',
});
var fortissimo = new TextDynamics({
    text: 'ff',
    duration: 'q',
});
var svorzando = new TextDynamics({
    text: 'sfz',
    duration: 'h'
});
var dynamics = score.voice([piano, fortissimo, svorzando]);

var notes = score.notes('a4/q, a4, a4, a4');
const system = vf.System({width: 300});
system.addStave({
    voices: [score.voice(notes), dynamics]
});

vf.draw();
```



StaveHairpin:

This is used to represent crescendos and decrescendos. These are fairly straightforward and similar to slurs. We specify a starting and ending note, and then a type. A type 1 is a crescendo, a type 2 is a decrescendo.

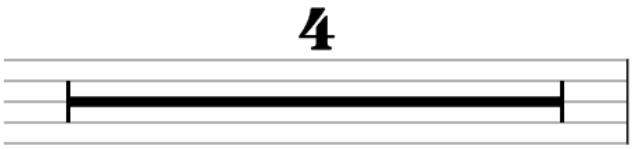
```
var notes = score.notes('a4/q, a4, a4, a4');
var system = vf.System({width: 300});
system.addStave({
    voices: [score.voice(notes)]
});

var hairpin = new StaveHairpin({
    first_note: notes[0],
    last_note: notes[3]
}, 1);
hairpin.render_options.y_shift = -110;
hairpin.setContext(vf.getContext());

vf.draw();
hairpin.draw();
```



MultiMeasureRest:

Our last feature, and luckily, these are very easy. There's no import required, it's built right off the factory. All we have to do is specify the number of measures we wish to have as rests and attach the stave:

```
const system = vf.System({width: 300});
var stave = system.addStave({
    voices: []
});

var rest = vf.MultiMeasureRest({number_of_measures: 4});
rest.setStave(stave);
```

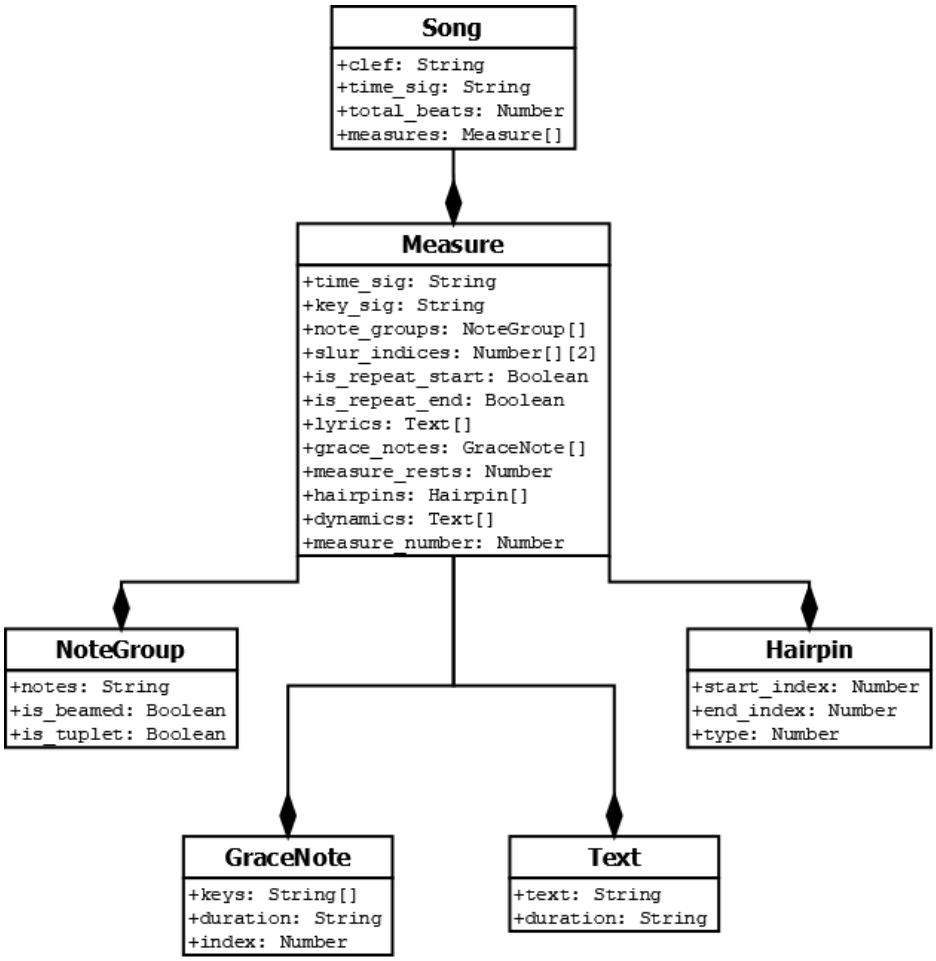


Note that any notes added to the stave voices will still be rendered.

Our Renderer:

Input Format:

The purpose of the renderer is to take output from our MusicXML parser in the form of a JSON file and render it on the screen. The following UML diagram outlines the structure of said file:



The three most important classes are the Song, the Measure, and NoteGroup classes. Together, they contain all the information needed to render a piece of music. The Song is a fairly simple object, operating mostly as a container for all the Measures, but it contains some extra information needed to render the very first Measure. The Measure itself is the largest class, containing all the information needed to render specific features within a Measure: time signature, key signature, repeats, etc. Most importantly, it contains an array of NoteGroups. The purpose of the NoteGroup class is to break up the notes in a Measure into several chunks where each chunk shares the same traits, i.e., tupleted, beamed, or none. This allows us to use EasyScore's functionality to apply these attributes to each chunk and then concatenate them together to form the full voice for the Measure. The other classes are fairly straightforward, as they are mostly recreations of the various VexFlow types we will be using, storing all the information needed to initialize a renderable version.

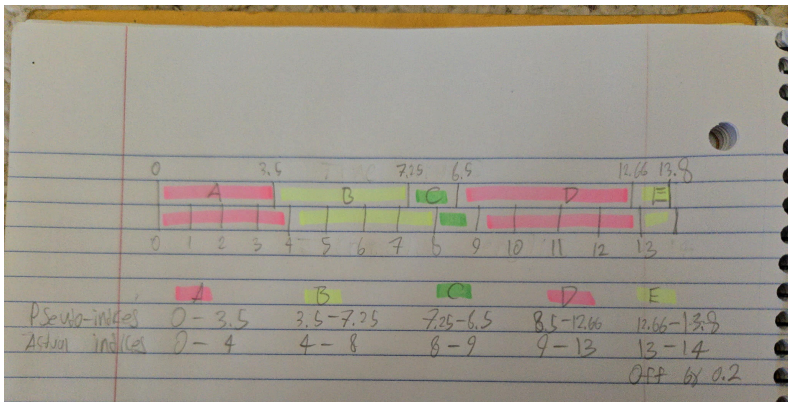
Outline:

- initialize(): Initialize Factory and EasyScore objects
- assembleMeasure(): create a System with all the Measure-specific information
    - assembleNotes(): combine NoteGroups into a single array of VexFlow notes
    - assembleGraceNotes(): attach any grace notes to the array of VexFlow notes

    - assembleLyrics(): combine Lyrics into an array of VexFlow Tickables
    - assembleDynamics(): combine dynamic markings (pp, p, f, ff) into an array of VexFlow Tickables
    - Construct voices out of the resulting Tickables
    - Create a System and Stave to render
    - addStaveModifiers(): add any modifiers (time signature, key, repeats) to the Stave
    - assembleSlur(): attach any slurs to the voice's Tickables
    - assembleHairpin(): attach any hairpins to the voice's Tickables
    - storePitches(): append the measure's pitches to an array for grading later
- Draw the system
- Draw slurs
- Draw hairpins
- Repeat for all other Measures

Grading:

While the renderer does not perform the grading itself, there are several features that have been designed to aid in the grading process. Firstly, we have set the width of each measure to be proportional to the number of beats that the measure contains. This, combined with setting the width of the rendered image to be fixed based on the number of beats in the total song, allows us to scroll the music at a fixed number of pixels per second, correlating accurately to a tempo in beats per minute. The other important feature is the array of pitches we keep track of. The app collects the user's pitch every 0.02 seconds and stores it in an array. For grading purposes, the renderer outputs a similar array. Given a note's duration, the tempo, and the increment of 0.02 seconds, we can calculate how many times that note would be appended to the array and construct a similar length/format array for the grader to compare against. The only issue is that the number of times a note should be added is not necessarily a whole number, so we have to smooth / round out the number of entries. The following diagram depicts the process better than I could explain it in words.



This ensures that we are at most only off by 1 index at all times, which, given the increment is 0.02 seconds, is miniscule.

Known Issues:

- Ties across bars are functional, but slurs across bars are not

- Grace notes are rendered too far to the left on notes that have multiple dots
- Slur parameters were largely designed through guess and check, and sometimes render a little strange
- Tuplets currently do not have bracket notation or work with partial beams (VexFlow does have support for brackets, but it was a detail beyond our scope)
- Tuplets of uneven durations are not supported, i.e., an eighth note and a sixteenth note under a tuplet to indicate a sixteenth note tuplet with a tie. VexFlow would render these as a tuplet of only two notes, not 3, so ticks may need to be manually adjusted
- Grace notes are hardcoded to always include a slash and a slur to the next note. This is not always desired, so future work may include this detail. They also do not work for multiple grace notes, we decided that was too small of a detail to focus on for the time being. It should be a relatively simple fix, both in the parser and the renderer, we just ran out of time.

Testing:

The renderer was tested using Mocha and Chai, which run in the browser. Unfortunately, VexFlow/EasyScore objects have unique IDs, which means that we cannot easily compare two objects for equality. Compounding the issue, objects are typically nested (sometimes circularly) within each other, making it inefficient to strip these identifiers before comparison. As a result, we have to compare the relevant attributes of each object, and have written helper functions to do exactly that. To run these tests, install npm, use npm to install mocha and chai, and change the “test”: tag in your package.json file to “mocha”. Then open the test HTML in the browser. The final note about testing is that several of the tests are designed to error. This means the console will display error messages, but they aren’t necessarily unintentional. Given that new unintended errors could pop up as new features are added, and more realistically as a result of the lack of time, there is no indication of which messages are intended and which ones are not, so I would suggest either determining a good way of distinguishing between them or at the very least becoming familiar with the messages when all tests pass as is. Most unintended errors will also cause the test to fail, so it shouldn’t be too problematic, but it’s good to be aware of.

Running in Browser:

Since the renderer does not currently run in the app, it’s essential to know how to run it in the browser. This is how the renderer was developed and tested, and the goal was to use WebView to embed this browser environment into the app, but we’ve run into a few problems. In a terminal, navigate to the folder containing the renderer, specifically whichever directory contains the HTML files. Then run: python -m http.server (or python3 -m http.server if using WSL). This will launch an HTTP server from the current directory, which you can then access in your browser. To use a different HTML file, just add /fileName.html at the end of the url. (e.g. localhost:8000/test.html)

Useful Tricks:

- Use console.log() early and often to print out objects and observe all their features. Sometimes it can be difficult to determine if an object is being created incorrectly, is not rendering, or is not being added to a stave. Printing the object out allows you to more easily tell what's going wrong, and given the lack of good documentation, is the easiest way to get an idea of what's really going on. It also gives you a better idea of how these objects are structured in the hierarchy.
- You can import all of the objects from Vexflow in one go using:

```
import * as Vex from "https://cdn.jsdelivr.net/npm/vexflow@4.2.2/build/esm/entry/vexflow.js";
const vf = new Vex.Factory({ renderer: { elementId: "output", width: 4000, height: 300 } });
```

  Just be sure to update any references with the name of the object you import as. (in this case 'Vex')
- Ticks can be confusing, especially when they're broken down into multiple separate categories, but here's a quick overview: ticks are fixed for each note duration, e.g. quarter notes are always 4096 ticks, no matter what time signature you're in. This value is found in the "intrinsicTicks" section of a note. It's the default ticks that the note contains. Sometimes, when notes are in a tuplet or dotted, they need to modify the ticks the note contains. This is done using a "tickModifier". For triplets, for example, the ticks for an eighth note are multiplied by two-thirds to get the actual duration. (Since multiplying by two gets the ticks for a quarter note and dividing by three makes it a triplet) The result of modifying the intrinsicTicks using a tickMultiplier is stored in the "ticks" section. This is essentially the final result, built from the other two sections, and is the value used to actually format the note.
- Note, this renderer was written using VexFlow 4.2.2, it may need to be updated as new features are added. As far as the documentation suggests, no features will break, but some naming conventions may need to be modified if moving to a new version.
- The code is commented fairly thoroughly, so I tried to tailor the contents of this documentation mostly to a broader overview of VexFlow and the general parts of the rendering process

Other Resources:

VexFlow Documentation
VexFlow Tutorial
EasyScore Tutorial
More Examples
Michael Toews (Feel free to shoot me an email if more help is necessary.)